

Introduction to Programming: Python

Mengzhuo Guo

Academic Year 2026-2027

Contents

1 Course Information	1
2 Course Objectives	1
3 Introduction of Python	2
4 Course Assessment and Grading	2
4.1 Attendance and Participation Policy	2
4.2 Assignment Policy	2
4.3 IMPORTANT: Academic Integrity and Use of AI Tools	3
5 Software and Learning Environment	3
6 Syllabus	3

1 Course Information

- **Course Name:** Python Language and Applications
- **Instructor:** Mengzhuo Guo
- **Class Time:** To be announced
- **Classroom:** To be announced
- **Contact:** mengzhguo@scu.edu.cn, or WeChat group

2 Course Objectives

Upon successful completion of this course, students should be able to:

- understand the fundamental syntax and concepts of Python;
- use conditional statements, loops, functions, and data structures to solve programming problems;
- apply exception handling, modules, and object-oriented programming in Python;
- independently design and develop simple Python programs;
- develop computational thinking and basic program debugging skills.

3 Introduction of Python

Python is a popular, high-level programming language. It is easy to learn because its syntax is simple, clear, and close to natural language.

Python supports object-oriented programming. Programmers can create classes and objects to organize code and make programs easier to reuse, maintain, and expand. It also supports other programming styles, such as procedural programming and functional programming.

Python is an interpreted and dynamically typed language, so programs can be written and tested quickly. It is also cross-platform, which means Python programs can run on Windows, macOS, and Linux. In addition, Python has a large number of libraries and frameworks for data analysis, artificial intelligence, web development, automation, and scientific computing.

Because of these features, Python is widely used by beginners, researchers, engineers, and software developers.

4 Course Assessment and Grading

The final grade for this course consists of attendance, individual assignments, and a final examination. The detailed grading scheme is shown in Table 1.

Table 1: Course Assessment and Grading Scheme

Assessment Component	Weight	Description
Attendance	10%	Students are expected to attend classes on time and actively participate in classroom activities.
Regular Grade	40%	Students are required to complete approximately five to six individual assignments during the semester. These assignments cover the main programming concepts, methods, and applications introduced in class.
Final Examination	50%	The final examination evaluates students' understanding and application of Python programming. The examination questions are mainly based on the individual assignments and course cases discussed during the semester.

The final course grade is calculated as follows:

$$\text{Final Grade} = 0.10 \times \text{Attendance} + 0.40 \times \text{Regular Grade} + 0.50 \times \text{Final Examination}.$$

4.1 Attendance and Participation Policy

Attendance accounts for 10% of the final grade. Attendance may be checked randomly during class. Students who are absent, late, or leave early without permission may lose attendance points.

Students are encouraged to participate actively in class. Those who answer questions, raise thoughtful questions, contribute to discussions, or help explain programming ideas *may receive additional participation points*.

Students who are unable to attend class for a valid reason should inform the instructor in advance and provide relevant documentation when required.

4.2 Assignment Policy

There will be approximately five to six individual assignments during the semester. Assignments must be submitted before the announced deadline.

Late submissions may receive a grade penalty unless prior permission has been granted. Students may discuss general ideas with classmates, but all submitted code and written explanations must be completed independently.

Submitted programs should run correctly in the required Python environment. Students are responsible for checking their files before submission.

4.3 IMPORTANT: Academic Integrity and Use of AI Tools

Students must complete all individual assignments independently. Copying code from classmates, websites, or other sources without proper acknowledgment *is not permitted*.

AI tools may be used to support learning, such as explaining concepts, identifying errors, or providing suggestions. However, students must understand, verify, and be able to explain all submitted code. Directly submitting AI-generated solutions without understanding or modification may be treated as academic misconduct.

The instructor may ask students to explain their code when necessary.

5 Software and Learning Environment

Students are *expected* to install the following software before the second class:

- Python 3.10 or later;
- Anaconda;
- Jupyter Notebook;

All assignments should be completed using Python 3 unless otherwise specified.

6 Syllabus

Table 2: Course Schedule and Useful Learning Resources

Class	Topic	Main Contents	Useful URLs
Class 1	Introduction to Python	Course introduction; applications and characteristics of Python; installation of Python and Anaconda; Jupyter introduction	Anaconda Download Introduction to Python Jupyter Documentation
Class 2	Fundamentals of Python	Basic syntax; variables and constants; identifiers; comments; input and output; basic operators and expressions.	Python Basics Python Installation Environment Python Development Tools
Class 3	Exercises on Python Fundamentals	Exercises covering variables, expressions, operators, input and output, and basic Python syntax.	Python Basic Exercises Jupyter Documentation
Class 4	Program Flow Control I	Boolean expressions; comparison and logical operators; conditional statements; <code>if</code> , <code>elif</code> , and <code>else</code> ; related exercises.	Python Control Flow
Class 5	Program Flow Control II	<code>for</code> loops; <code>while</code> loops; <code>break</code> , <code>continue</code> , and <code>pass</code> ; nested loops and related exercises.	Python Control-Flow Tools Python Basic Tutorials
Class 6	Common Built-in Data Types I	Numbers; Boolean values; strings; string indexing, slicing, formatting, and common string methods; related exercises.	Python Built-in Types Python Strings
Class 7	Common Built-in Data Types II	Lists and tuples; indexing and slicing; common methods; mutable and immutable objects; related exercises.	Python Lists Python Data Structures

Continued on next page

Table 2 (continued)

Class	Topic	Main Contents	Useful URLs
Class 8	Composite Data Types I	Dictionaries and sets; creation, access, modification, deletion, traversal, and commonly used methods; related exercises.	Dictionaries and Sets
Class 9	Composite Data Types II	Nested data structures; conversion between data types; selection of suitable data structures; list comprehensions and comprehensive exercises.	Python Data Structures List Comprehensions
Class 10	Errors and Exception Handling	Syntax errors and runtime errors; try , except , else , and finally ; raising exceptions and designing exception-handling programs.	Errors and Exceptions
Class 11	Functions I	Function definition and invocation; return values; positional arguments; keyword arguments and default arguments; related exercises.	Defining Functions
Class 12	Functions II	Variable-length arguments; variable scope; local and global variables; recursive functions; documentation strings and related exercises.	Function Arguments and Scope
Class 13	Functional Programming and Exercises	Lambda expressions; functions as objects; map , filter , and reduce ; comprehensive exercises on functions.	Lambda Expressions Functional Programming
Class 14	Object-Oriented Programming I	Basic concepts of object-oriented programming; classes and objects; attributes and methods; constructors and the self parameter; related exercises.	Python Classes
Class 15	Object-Oriented Programming II	Encapsulation; inheritance; method overriding; polymorphism; class attributes and instance attributes; related exercises.	Inheritance and Polymorphism
Class 16	Modular Programming	Modules and packages; importing modules; import and from ; the <code>__name__</code> variable; creating and using custom modules.	Python Modules and Packages
Class 17	Course Review	Review of Python syntax, flow control, data types, functions, exception handling, object-oriented programming, and modular programming; comprehensive exercises.	Python Tutorial Python Standard Library